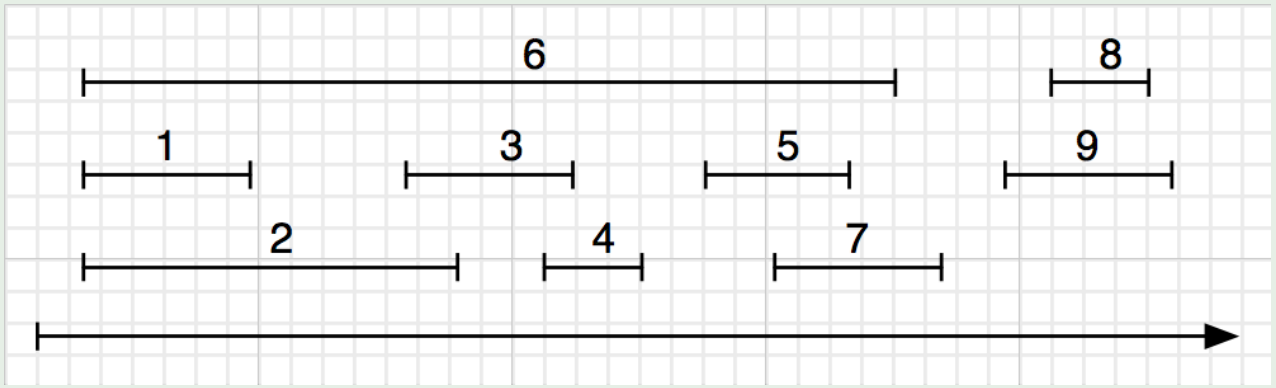
Designing a greedy algorithm

Definition (Interval scheduling problem)

Given a set of schedules $I = \{I_i\}$, find the largest set $A \subseteq I$ such that the members of A are non-conflicting.

Example

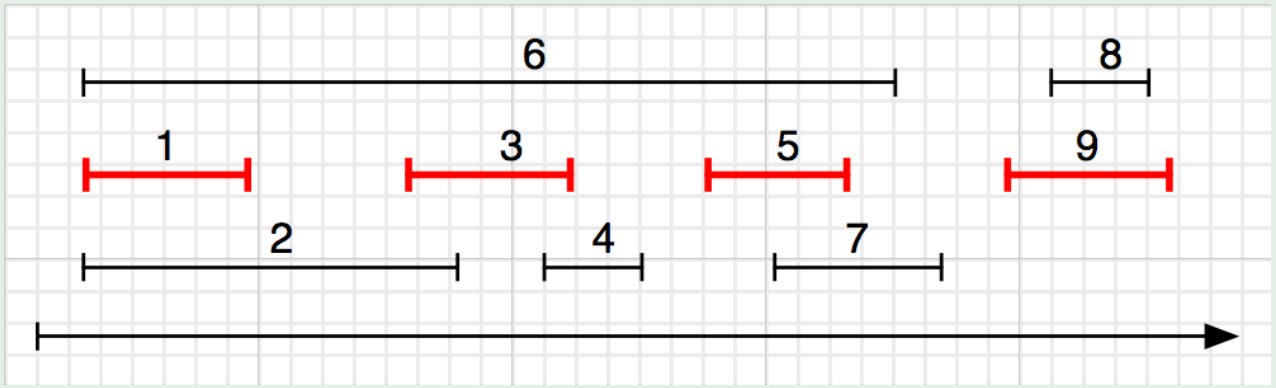


Designing a greedy algorithm

Definition (Interval scheduling problem)

Given a set of schedules $I = \{I_i\}$, find the largest set $A \subseteq I$ such that the members of A are non-conflicting.

Example



$A = \{1, 3, 5, 8\}, \quad |A| = 4.$

Question

How about $\{2, 4, 7, 8\}$? $\{1, 3, 5, 9\}$?

Brute force solution

A solution must be one of the subsets of the set of intervals.

1. Enumerate all possible *configurations* (i.e., all possible subsets of the intervals).
2. Go through the set of subsets and remove the ones that have one or more conflicting schedules.
3. Pick (any one of) the largest subset from the ones that survive.

Complexity

- There are $2^n - 1$ non-empty subsets, one or more of which may be a feasible solution.
- Each feasible solution must be scanned for conflict, which takes $O(n)$ time.
- The algorithm runs in $\Theta(n2^n)$ time $\Rightarrow$ an exponential time algorithm!

Designing a greedy algorithm (continued)

Basic steps

To compute the maximal set of intervals that can be scheduled, the basic idea is to:

1. Use a "simple" rule (or strategy) to select the first interval i_1 to be accepted.
2. Once i_1 is accepted, remove from consideration all intervals the conflict with i_1.
3. Select the second interval i_2 to be accepted, and remove all the intervals that conflict with i_2.
4. And so on until there are no more requests remain.

Key challenge

How to choose the "simple" rule to select the next interval that leads to an optimal solution?

Designing a greedy algorithm (continued)

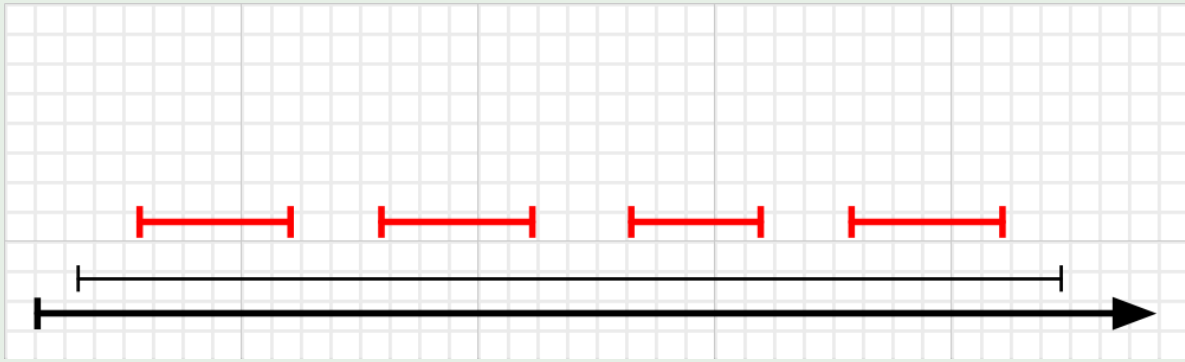
Strategy 1. *Earliest First*

The idea is to start using the resource as early as possible.

1. Sort the intervals by starting time, breaking ties arbitrarily.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

This strategy does not lead to an optimal solution.

Example (using an optimal strategy)



$|A| = 4$.

Designing a greedy algorithm (continued)

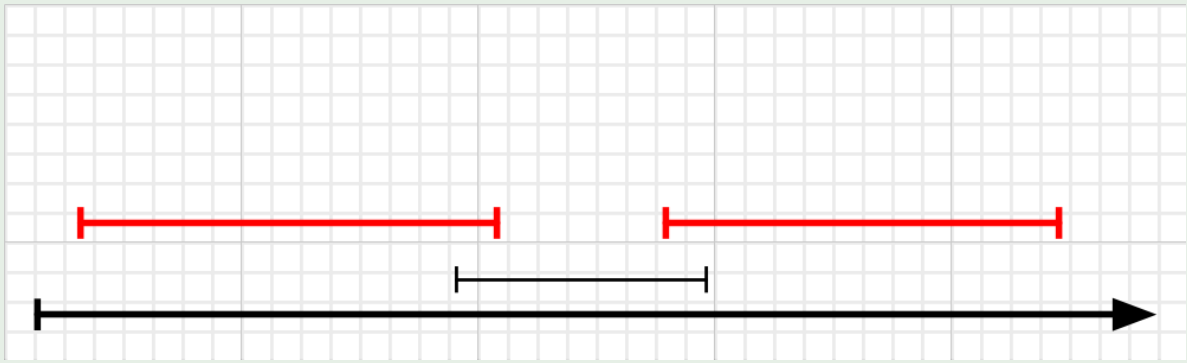
Strategy 2. *Shortest First*

The *Earliest First* strategy failed perhaps because it missed the shorter intervals, which would accommodate more intervals.

1. Sort the intervals by length, breaking ties arbitrarily.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

This strategy does not lead to an optimal solution.

Example (using an optimal strategy)



$|A| = 2$.

Designing a greedy algorithm (continued)

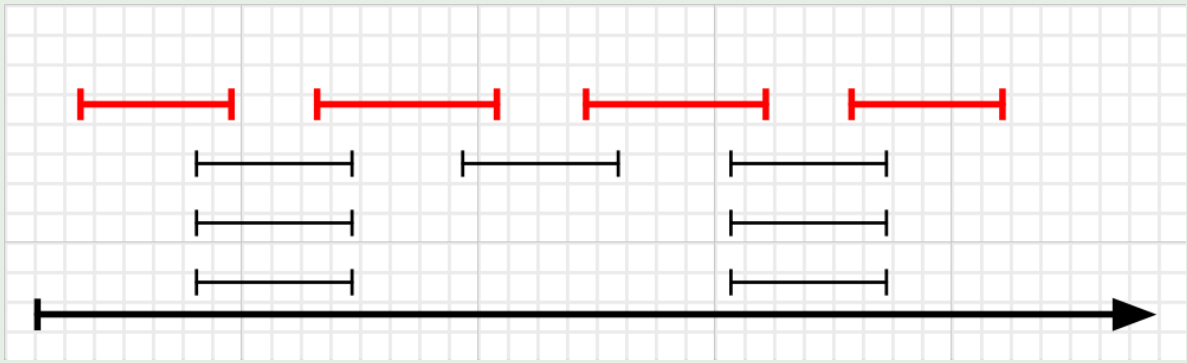
Strategy 3. *Least-conflict First*

The *Shortest First* strategy failed perhaps because the shorter ones had more conflicts, and ruled out too many intervals in the process.

1. Sort the intervals by the number of other intervals which conflict with it.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

This strategy does not lead to an optimal solution.

Example (using an optimal strategy)



$|A| = 4$.

Designing a greedy algorithm (continued)

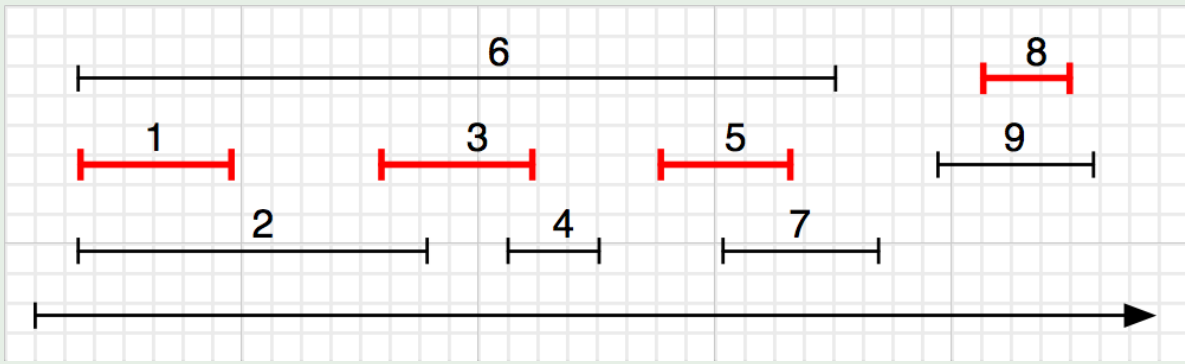
Strategy 4. *Finish First*

The idea is to free up the resource as early as possible.

1. Sort the intervals by the finishing time, breaking ties arbitrarily.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

This strategy is the one that works. But can you prove that it works?

Example (using optimal *Finish First* strategy)



$|A| = 4.$

An $O(n \lg n)$ greedy algorithm for interval scheduling

SCHEDULE-INTERVALS(I) $\quad \triangleright\ I = \{I_i\}, I_i = (s_i, f_i)$

```
1  R = Sorted requests in order of finishing times such that f_i ≤ f_j when i < j.
2  Create an array S[1..n] with starting times such that S[i] contains s_i.
3  A = {R_1}                                   ▷ select first interval from sorted list
4  f = f_1
5  while there are more intervals in S to look at
6        do j = first interval for which s_j ≥ f
7           A ← A ∪ {j}
8           f ← f_j
9  return A
```

Analysis

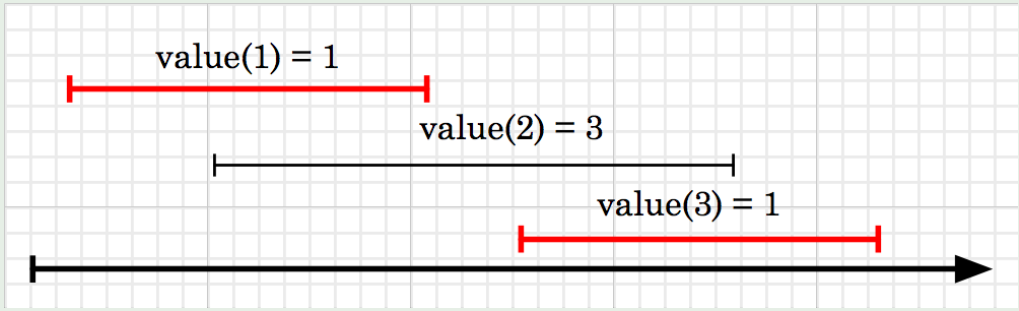
- The sorting step in takes $O(n \lg n)$ time.
- Creating the starting time array $S[1..n]$ takes $O(n)$ time.
- The single pass through the array S takes $O(n)$ time
- An $O(n \lg n)$ time algorithm for a problem with a natural search space of $O(n2^n)$.

Extension: weighted interval scheduling problem

Definition (Weighted interval scheduling problem)

Given a set of schedules $I = \{I_i\}$, with associated weights $W = \{w_i\}$, find $A \subseteq I$ such that the members of A are non-conflicting *and* the total weight $\sum_{i \in A} w_i$ is maximized.

Example (using our greedy strategy)



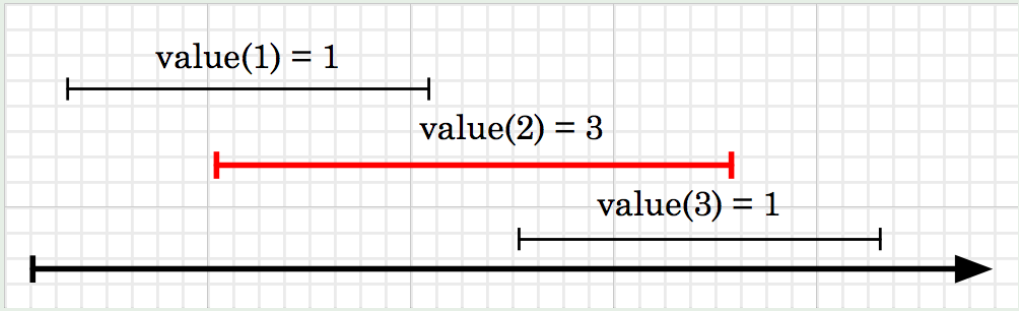
$$|A| = 2, \sum_{i \in A} w_i = 2.$$

Extension: weighted interval scheduling problem

Definition (Weighted interval scheduling problem)

Given a set of schedules $I = \{I_i\}$, with associated weights $W = \{w_i\}$, find $A \subseteq I$ such that the members of A are non-conflicting *and* the total weight $\sum_{i \in A} w_i$ is maximized.

Example (using an optimal strategy)



$$|A| = 1, \sum_{i \in A} w_i = 3.$$

Hmmm. . .

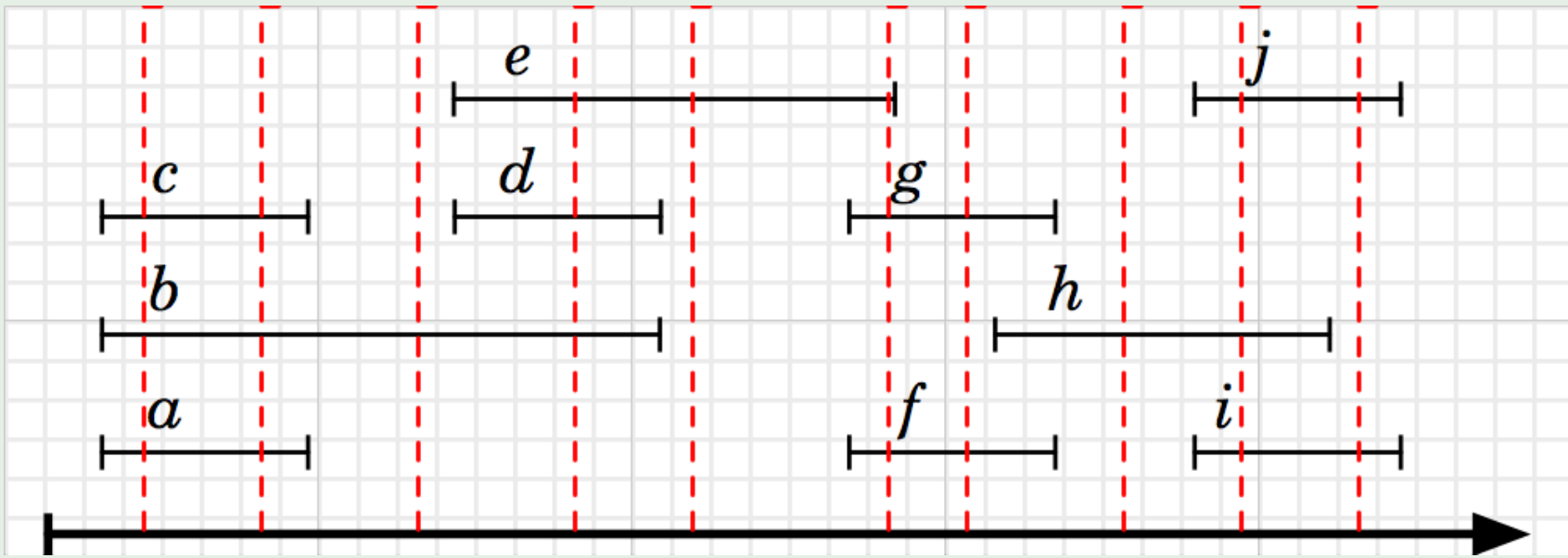
There is no greedy solution for the weighted interval scheduling problem!

Scheduling all intervals greedy algorithm

Definition

Given a set of schedules $I = \{I_i\}$, find the minimum number of resources needed to schedule I such that the intervals on each resource are non-conflicting.

Example



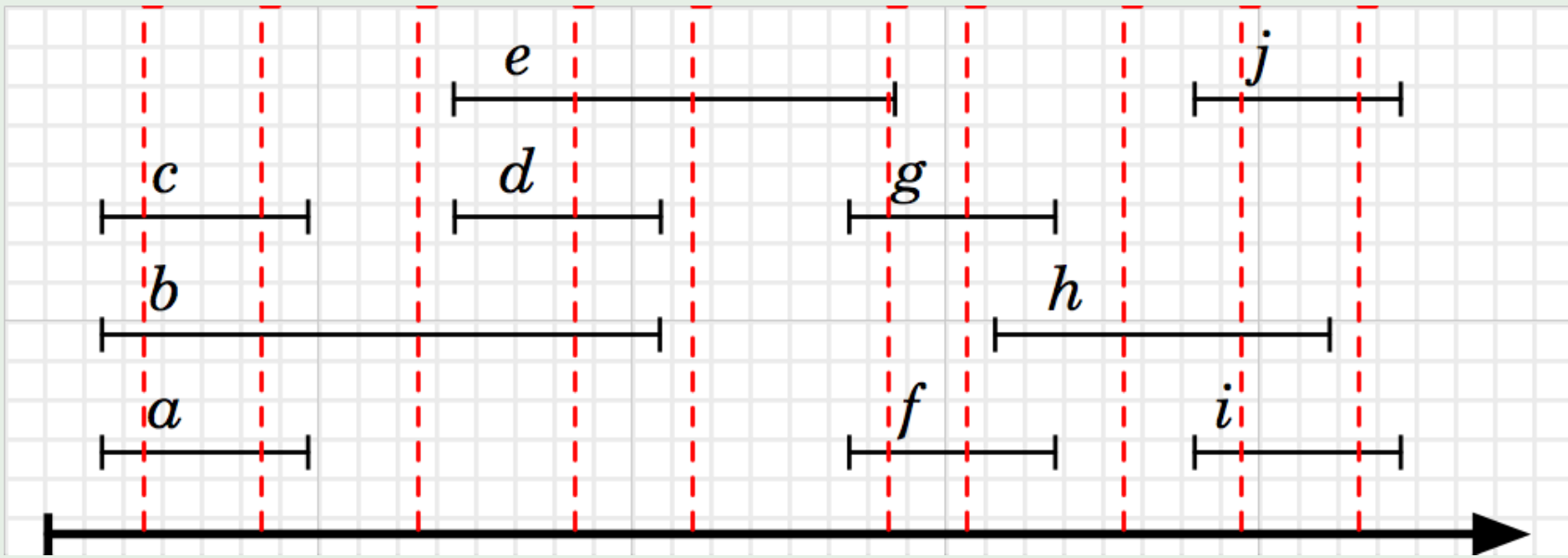
Depth = Maximum number of intervals at any point in time.

Scheduling all intervals greedy algorithm

Definition

Given a set of schedules $I = \{I_i\}$, find the minimum number of resources needed to schedule I such that the intervals on each resource are non-conflicting.

Example



Depth $= 3 \implies$ Minimum # of resources needed $= 3$

A greedy algorithm for scheduling all intervals

SCHEDULE-INTERVALS(I) $\quad \triangleright\ I = \{I_i\}, I_i = (s_i, f_i)$

1 $R =$ Sorted requests in order of starting times, breaking ties arbitrarily, such that $s_i \leq s_j$ when $i < j$.
2 $m \leftarrow 0$ $\triangleright$ the optimal number of resources needed to schedule R
3 **while** $R \neq \emptyset$
4 **do** $req =$ extract the next element in R
5 **if** there is a resource j with no interval conflicting with req
6 **then** schedule interval req on resource j
7 **else**
8 $m \leftarrow m + 1$ $\triangleright$ allocate a new resource
9 schedule interval req on resource m

Complexity

$T(n) = O(n \lg n)$. Prove it.